FSP Talent Market — техническая документация

Версия 2026-10-11. Репозиторий: приватный GitHub (доступ выдаёт владелец). Демо-данные синтетические; результаты валидации не являются оценкой точности на реальной популяции.

Product

Thesis: reverse recruiting on verified evidence. Not a job board, not résumé search, not a résumé scorer.

Candidate: profile → specialisation + claimed grade → parameterised Core Assessment → `VERIFIED` / `NOT_VERIFIED` → Evidence Profile (claims vs evidence, FSP evidence) → addressed invitations with a visible salary → Accept / Decline.

Employer: need → hard constraints → Requirement Fit with WHY / GAPS → candidate view (no contacts) → salary-backed invitation → contacts only after the candidate accepts. A second mode, Full Pool, lists the whole visible bank with filters.

Central abstraction: **Evidence Profile** — `Skill Claim → Skill Evidence → Verified Skill Level`; résumé data is context, FSP achievements are separate external evidence, and there is no universal Talent Score. "Fit" always means fit to one concrete need.

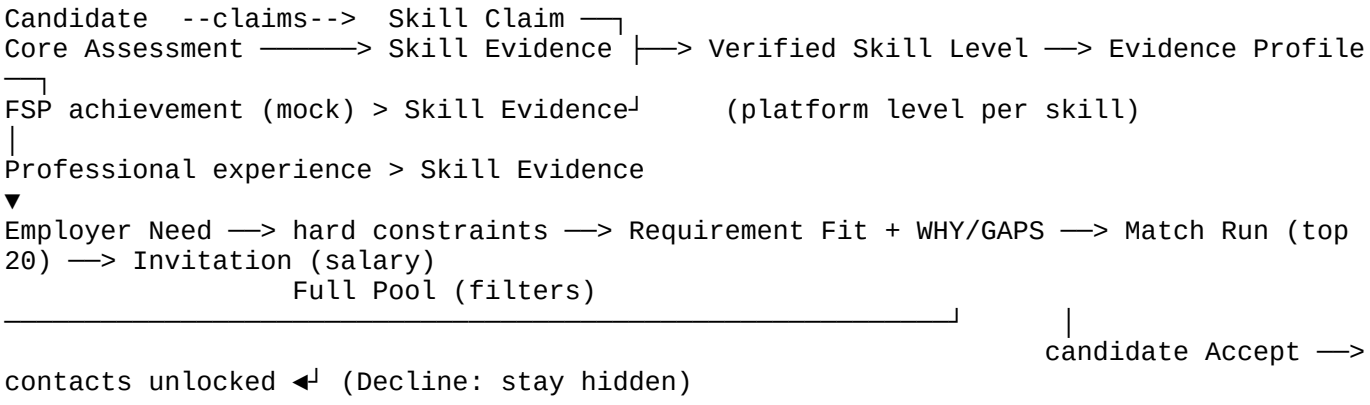
MVP scope: Backend / Python fully implemented (Junior / Middle / Senior, nine sublevels J1–S3); other specialisations exist only as taxonomy rows. Mock FSP provider. Roles: candidate, employer.

Non-goals (not built): calibrated IRT, arbitrary code execution sandbox, LLM judging, résumé parsing, universal score, ATS, anti-fraud, chat, vacancy marketplace, real Keycloak / FSP API, email delivery.

Architecture

Modular monolith: **React + TypeScript** (Vite, served by nginx) → **FastAPI** → **PostgreSQL** (SQLAlchemy 2, Alembic). Docker Compose runs `db`, `backend`, `frontend`. No queues, no Redis, no microservices, no LLM dependency; CPU-only.

Functional architecture



Categorisation: a candidate's **category** is `specialization × public grade` (e.g. *Backend / Middle*), with internal sublevels J1 J2 M1 M2 S1 S2. Only Backend / Python is fully implemented; other specialisations stay in the taxonomy as data (a frontend specialisation row exists as a seed example). The grade is set only by the Core Assessment.

Components (`src/fsp_market/`)

module	responsibility
`api/`	FastAPI routers (candidate, employer, invitations, demo, validation), dependencies, role checks
`auth/`	HMAC-signed bearer tokens with a role claim; swappable behind the dependency for Keycloak
`taxonomy/`	specialisations, competencies, grade↔sublevel helpers
`candidates/`, `employers/`	entities; candidate preferences (salary, formats, visibility, consent flags)
`skills/`	`SkillClaim` (what is claimed), `SkillEvidence` (what supports it), `VerifiedSkillLevel`
`assessments/`	Family → Template → Preset → Variant, attempts, answers, results, scoring, seed
`evidence/`	evidence creation (Core Assessment, FSP, experience), Evidence Profile composition
`matching/`	policy, pure engine (constraints, fit, WHY/GAPS), match runs, Full Pool, employer-safe views
`invitations/`	invitation model, state machine, the only place that releases contacts
`fsp/`	`FSPProvider` interface, `MockFSPProvider`, discipline→skill mapping
`validation/`	fixed dataset, labels, baselines, metrics, assessment reference validation
`demo.py`	demo seed / reset

Data model (tables)

`specializations`, `competencies`, `skills`, `candidates`, `employers`, `skill_claims`, `skill_evidence`, `verified_skill_levels`, `task_families`, `task_templates`, `task_presets`, `task_variants`, `assessment_attempts`, `assessment_answers`, `assessment_results`, `employer_needs`, `need_skills`, `match_runs`, `match_results`, `invitations`. Migrations `0001`–`0004` in `src/fsp_market/migrations/versions/`. Variants store `seed` and `is_anchor`, results store sub-scores and rubric results (IRT-ready storage; **no calibration is claimed**).

Invariants enforced in code (and covered by tests)

- Platform grade changes only through `score_attempt`; résumé, FSP, needs and fit never write it.
- A failed attempt sets `NOT_VERIFIED`, keeps the claimed grade, never lowers or invents a level.
- FSP evidence has no level; it cannot create or raise a `VerifiedSkillLevel`.
- Employer-facing views contain no email, phone, full name or FSP member id; contacts exist only in `invitations.service.contacts_for` for `ACCEPTED`/`CONTACTS_UNLOCKED` invitations of that employer.
- Salary range and GROSS/NET are validated server-side on every invitation.

Deployment

`docker compose up --build`: backend image runs `alembic upgrade head`, then `python -m fsp_market.demo seed --if-demo-mode`, then uvicorn; nginx serves the SPA and proxies `/api`, `/health`, `/docs`, `/openapi.json`. Configuration (all optional, see `.env.example`): `DATABASE_URL`, `JWT_SECRET`, `ENVIRONMENT`, `DEMO_MODE`. With `ENVIRONMENT=production` the app refuses to start if `DEMO_MODE` is on or the token secret is a placeholder.

Dependencies

Python 3.12; fastapi 0.142.2, starlette 1.7.0, uvicorn 0.54.0, sqlalchemy 2.1.3, alembic 1.20.0, psycopg 3.3.6, pydantic 2.13.5, pydantic-settings 2.15.0 (pinned in `constraints.txt`; dev: pytest, httpx, ruff). Frontend: react 18.3.1, vite 5.4.21, typescript 5.9.3, vitest 2.1.9 (locked in `package-lock.json`). Images: postgres:16-alpine, nginx:1.27-alpine, node:22-alpine, python:3.12-slim.

Core Assessment mechanics (Backend / Python)

The Core Assessment is the **only** source of the platform grade.

Structure

`Competency → Task Family → Task Format → Task Template → Parameter Preset → Task Variant → Fixed Scoring Contract`

- **Task Family** fixes competency, target grade, difficulty band, rubric (dimensions + weights), edge-case classes, allowed formats.
- **Task Template** fixes prompt structure, format, parameter schema, scoring rule.
- **Parameter Preset** is a hand-verified parameter set (no free-form generation, no LLM).
- **Task Variant** is a concrete instance: prompt, options, `seed`, expected behaviour, scoring contract, `is_anchor`.
- Variants of one family change domain, schema, endpoint, numbers or scenario; they keep competency, difficulty, rubric dimensions and scoring contract. Rendering is a pure function (`assessments/variants.py`): same template + preset + seed ⇒ same variant.

Seed content (rubric `backend-core-2026.10.2`): six competencies (Python, SQL, API/FastAPI, Testing, Docker basics, System thinking), **11 task families per grade** (33 total), 6–10 verified presets (variants) per family; 11 tasks per attempt (time limits 15 / 19 / 23 min for Junior / Middle / Senior). Formats used: code reading, SQL reasoning, API reasoning, deterministic/rubric questions (single choice). No arbitrary code execution.

Flow and scoring

1. `POST /api/assessments {target_grade}` — the candidate's **own claim**; creates (or returns the existing) in-progress attempt. One variant per family is assigned: `variants[attempt_id % n]` — reproducible from the attempt id.

2. The client sees prompts and options only; the correct option never leaves the server (except the demo-only helper below).

3. `POST /api/assessments/{id}/submit` — must answer exactly the assigned tasks with valid options (else 422).

4. Scoring is exact-match and deterministic. Total score = earned/possible points. Pass thresholds: **Junior 0.60, Middle 0.70, Senior 0.75**; the upper sublevel (J2/M2/S2) needs ≥ 0.85.

5. Result `VERIFIED` (platform level + sublevel set) or `NOT_VERIFIED` (claimed grade kept, nothing lowered).

A competency with score ≥ the grade's threshold creates `CORE_ASSESSMENT` evidence (strength HIGH) and a `VerifiedSkillLevel` for the matching skills (never lowering an existing higher level).

6. A failed attempt blocks the same target grade for **7 days** (anti grade-hopping; MVP policy, not calibrated). Another grade can be attempted at any time.

Why it resists simple answer passing

- Different attempts get different variants of the same family; surface data, options and the position of the correct option differ, so a memorised "A, C, B…" sheet or a pasted answer to one variant does not carry over to its sibling.
- Answer keys are server-side only; submissions are validated against the assigned set; failed targets have a cooldown, so brute force by retries is slow.
- Honest limits: each family has 6–10 variants, so collusion resistance is moderate; the parameter-preset design is meant to scale to many more; there is no proctoring or plagiarism detection (roadmap). Attempts are 11 items; the guessing baseline is simulated in the validation results.

Comparability and "no calibration" statement

Variants are comparable by the shared family contract, fixed rubric, an anchor task and validation — **not** by psychometric calibration. Data needed for later IRT is stored (family, template, variant, candidate, grade, sub-scores, timestamps) but no IRT/2PL model is fitted or claimed.

Demo-only helper

With `DEMO_MODE=true`, `GET /api/demo/assessments/{id}/reference-answers` returns the correct options of the caller's **own** attempt so a jury can click through quickly. It returns 404 when demo mode is off and must never be enabled in production.

Matching mechanics

Two employer modes share the same candidate data and privacy rules.

Mode 2: need-based matching

`Employer Need → Hard Constraints → Requirement Fit → ranked Match Run → WHY / GAPS`

A **Need** has role, specialisation, target grade (+ optional sublevel), required and optional skills (default weights 1.0 / 0.4, stored per need), salary range with GROSS/NET, acceptable work formats, optional city and latest start (days), description. No LLM parsing in P0.

Hard constraints (before any scoring)

constraint	rule
specialisation	must equal the need's
grade	**internal sublevel ±1**: target M2 → {M1, M2, M3}, target M3 → {M2, M3, S1} (crosses the grade border); without a target sublevel only the grade's own three sublevels. Verified candidates use their verified sublevel; others the whole range of the claimed grade. Never ±1 public grade
salary	candidate minimum ≤ need maximum (inclusive); NET↔GROSS converted at 13 % (policy

	parameter); unknown expectation does not exclude
work format	at least one shared format
location	only checked when the shared formats exclude REMOTE
availability	candidate start ≤ need's latest start, when both are set
visibility / consent	profile visible and open to invitations

Requirement Fit

`fit = Σ wᵢ·cᵢ / Σ wᵢ` over active components, × 0.6 for candidates whose grade is not VERIFIED (kept in the pool, ranked lower). Default weights (`matching/policy.py`, one place, snapshotted into every run): skill coverage 0.40, assessment 0.20, professional experience 0.15, relevant FSP 0.10, freshness 0.10 (a 0.05 semantic "context" slot exists but is inactive: no LLM). These are **initial tunable heuristics**, not a scientifically derived formula.

Skill credit: verified at or above the need's grade = 1.0; verified below = 0.5; professional experience only = 0.5 (reported as `WEAK_EVIDENCE`); claim only or nothing = 0. FSP counts only through skills relevant to the need. The result is shown as **"Соответствие этой потребности: XX %"** — fit to this need, never a candidate score.

WHY / GAPS

Each reason has a machine code and Russian text, built only from facts in the data: WHY — `GRADE_VERIFIED`, `SKILL_VERIFIED`, `SKILL_EXPERIENCE`, `SALARY_OVERLAP`, `WORK_FORMAT_MATCH`, `FSP_RELEVANT`; GAPS — `GRADE_NOT_VERIFIED`, `SKILL_UNVERIFIED`, `SKILL_BELOW_TARGET`, `WEAK_EVIDENCE`. Works fully without an LLM.

Match Run

`POST /api/employer/needs/{id}/match` stores an immutable `MatchRun` with the policy snapshot, the exclusion summary and the top 20 `MatchResults` (stable order: fit ↓, verified first, candidate id). Reading a run supports `limit/offset`, `min_fit`, `verified_only`, `skill` without changing it; the UI shows the first 5 and "show more".

Mode 1: Full Pool

`GET /api/employer/pool` — all *visible* candidates with filters: specialisation, platform grade, sublevel, verification status, verified skills, FSP history, work format, city, salary expectation. `NOT_VERIFIED` candidates stay findable.

Privacy

Employer views (`matching/service.py: public_candidate`) show an abbreviated name, levels, verified skills, evidence summary, salary expectation. No email, phone, full name or FSP member id. Contacts appear only through an accepted invitation (see `invitations/`).

ATS export

`GET /api/employer/match-runs/{run_id}/export?format=csv|json` returns the stored run (rank, candidate id, abbreviated name, grades, fit, formats, location, salary expectation, verified skills, WHY/GAP codes). Owner-only; no contacts. CSV has a UTF-8 BOM and neutralises formula-injection prefixes.

Reproducibility and invitation rules (2026-10-11)

- A match run is evaluated at the start of the UTC day and stores `policy.input_fingerprint` (hash of the need spec and every candidate's facts). Same fingerprint ⇒ identical composition, order, WHY and GAPS; a changed pool or candidate state changes the fingerprint, so differences are explainable. Tie-break: Requirement Fit desc, VERIFIED first, candidate id.
- Experience is a weak, unconfirmed signal (context fit weight 0.05 and 50% skill credit, always labelled "not confirmed"); only the Core Assessment assigns a grade.
- An invitation for a saved need is validated server-side: role, salary type, salary range and work format must stay within the need; the candidate must pass the need's hard filters and the offered range must reach the candidate's stated minimum. A standalone invitation (Full Pool) has its own validation and is tracked separately from need-scoped ones: uniqueness is per employer + candidate + need.
- A CLOSED need is excluded from matching and new invitations; its runs and sent invitations remain. Needs are shown to candidates only when published by the employer.
- A candidate profile is visible to employers only if the visibility intent is on **and** the email is verified **and** invitations are open.

Matching and assessment validation (MVP)

This is MVP validation, not a psychometric calibration. The organisers did not provide an official

dataset or ground truth, so we use our own fixed, reproducible validation set. All numbers quoted anywhere

come from `artifacts/validation/latest.json` / `latest.md`, which are produced by code; nothing is typed in by hand.

Run it

```
.venv/bin/python -m fsp_market.validation            # runs everything, writes
artifacts/validation/latest.{json,md}
.venv/bin/python -m fsp_market.validation --check    # re-runs and fails if the
stored artifacts differ
```

Needs no database server (in-memory SQLite), no network, no LLM. The same data and code give byte-identical

`latest.json` (verified across separate processes; the file has no timestamps). Artifacts must be regenerated and

committed whenever the data, labels, matching policy or assessment content change

(`tests/test_phase6_validation.py` fails if they are stale).

Where the data comes from

`src/fsp_market/validation/data/` (version-controlled):

file	content
`candidates.json`	32 hand-designed profiles: claimed grade, assessment behaviour, salary, formats, location, experience, résumé keywords, FSP disciplines, a note on why the profile exists
`needs.json`	10 employer needs (grade/sublevel, required and optional skills, salary + GROSS/NET, formats, location, start limit)

`relevance.json` hand-made relevance labels (0-3) with a tag explaining each

Profiles are loaded through the real pipeline: verified candidates take the actual Core Assessment (their answers are

fixed per profile), then evidence, FSP mapping and matching facts are created by the production code. Conflict cases:

strong CV + failed test, weak CV + strong test, relevant and irrelevant FSP evidence, salary mismatch (also exact

boundary and NET), work-format mismatch, NOT_VERIFIED Middle, no FSP history, missing required skill, strong optional

skills but a failed hard constraint, hidden / closed profiles, other specialisation, availability.

How the labels were made

Scale: 0 irrelevant (fails a hard constraint or has no relevant evidence), 1 partially relevant, 2 relevant,

3 highly relevant. Each non-zero label carries a rubric tag (see `tags` in `relevance.json`). They were assigned by

hand from the dataset facts and written **before** the matching code was run on this dataset; the matching score was

never an input. Pairs not listed are 0. Relevant = grade >= 2. Limitation: one team wrote both the rubric and the

matching policy, and the rubric follows similar logic (see below).

Methods compared (same candidates, same needs, same labels)

1. **Keyword baseline** - résumé keyword search over the candidate's keyword list: required keyword 2 points,

optional 1, ties by id. No constraints and no verified evidence.

2. **Hard filters only** - the production hard constraints, survivors in candidate-id order (no ranking).

3. **Evidence matching** - the production pipeline: hard constraints, then Requirement Fit

(skill coverage, assessment, professional experience, relevant FSP evidence, freshness), with NOT_VERIFIED

candidates kept but penalised.

Metrics

Per need, macro-averaged over needs: Precision@3, Precision@5 (denominator k), Recall@10, nDCG@10 (gain 2^g-1,

discount log2(rank+1), ideal ordering from all labels). *Hard-constraint violation rate*: share of the returned

top-10 that violates a constraint according to an **independent** checker (`validation/constraints.py`, a second

implementation written from the dataset fields, not from the engine). Ordering determinism: two runs compared.

Sensitivity: three weight configurations fixed in advance (default, assessment-heavy, FSP-heavy); top-5 overlap and

top-1 agreement versus default. Weights were not tuned on the labels.

Assessment validation

Seven synthetic reference personas (Junior-like, Middle-like, Senior-like, weak, and boundary cases) answer by a fixed

rule; the expected outcome per persona and grade is written by hand from the thresholds and item counts. Each case runs

on both equivalent variants of every task family. Reported: grade accuracy, highest-grade accuracy, variant stability

(identical outcome and score difference between equivalent variants), disagreement of more than one sublevel,

discrimination between personas that know a grade and those that do not, competency coverage per grade, rubric/content

integrity checks, anchors, and a seeded random-guess false-verification rate. This is **reference-solution

validation**: it checks rules, thresholds and content, and says nothing about real candidates or test-retest reliability.

Known limitations

Listed in `latest.md` (generated). In short: own dataset and labels, small and designed set, perfect-fit candidates

compress differences, label/engine similarity, simple keyword baseline, Junior (2 items) and Senior (4 items) tests are

short and guessable at a visible rate, single-choice items only.

Validation

Reproducible matching + assessment validation lives in [`VALIDATION_METHOD.md`](VALIDATION_METHOD.md) (method, data, labels, baselines, metrics, limitations). Results are generated, not typed: `artifacts/validation/latest.{json,md,summary.md}` and the generated block in the root `README.md`. Run: `python -m fsp_market.validation` (`--check` to verify reproducibility).

This is MVP validation on our own dataset (the organisers provided none), not a psychometric calibration.

FSP integration

No production FSP API or FSP ID stand was available, so the MVP integrates through an interface and a mock. Real access changes only the adapter, not the product.

```
FSPProvider (Protocol, fsp/provider.py)
├── get_member(fsp_id)        -> FSPMember | None
```

```
├── get_achievements(fsp_id)  -> list[FSPAchievement]
└── verify_member(fsp_id)     -> bool
```

`MockFSPProvider` serves fixtures: `FSP-1001` (verified member with 3 achievements: algorithmic, product programming, cybersecurity), `FSP-2002` (verified member **with no history**), `FSP-9999` (unverified member, rejected). Unknown ids return no member and no achievements; it never raises.

From achievement to evidence

`evidence/fsp_link.py` verifies the member, then maps each achievement's discipline to the relevant skills (`fsp/mapping.py`: algorithmic → python; product programming → api_fastapi, sql, testing; cybersecurity → none in the Backend track) and stores one `FSP_ACHIEVEMENT` evidence row per skill (strength MEDIUM, no level, details: achievement id, discipline, year, place). Linking is idempotent.

Guarantees: FSP evidence **never changes the grade** or any `VerifiedSkillLevel`, adds no points, and counts in matching only for skills relevant to the need. A candidate with no FSP history (or an empty linked profile) works exactly like any other; the Evidence Profile shows an explicit empty state.

Replacing the mock

1. Implement `FSPProvider` against the real FSP API / FSP ID (OIDC via Keycloak) and register it in `api/deps.py: get_fsp_provider`.

2. Replace the bearer-token issuing (`auth/tokens.py`, registration) with Keycloak-issued tokens behind `current_candidate_id` / `current_employer_id`; keep the role claim.

3. Extend `DISCIPLINE_SKILLS` for new disciplines; no change to the matching or grading code.

If the organisers publish a test stand or authorisation requirements, only steps 1–2 are affected.

Decisions

- D1 Modular monolith over PostgreSQL; the evidence "graph" is relational. No queues, Redis, microservices or LLM dependency.
- D2 Deterministic grading and matching; no LLM decides grades, hard constraints, verification or contact unlock.
- D3 IRT-ready storage, no calibration claims.
- D4 Platform grade comes only from the Core Assessment. A failed attempt gives `NOT_VERIFIED` and keeps the claimed grade; there is no automatic downgrade. Same-target retry after a 7-day cooldown (MVP policy, not calibrated).
- D5 FSP is external evidence behind `FSPProvider` (mock in MVP); it never changes the grade; candidates without FSP history are first-class.
- D6 No arbitrary-code sandbox in the MVP (P0 = deterministic single-choice items: code reading, SQL/API reasoning); a gVisor-backed runner is roadmap.
- D7 Implementation used a Factory/Bonsai coding worker where it was fast; from Phase 2 the Factory API needed a bearer token that was not available, so the local Bonsai endpoint was called directly, and the controller wrote the modules where Bonsai stalled or produced incorrect code. Architecture was not affected.
- D8 Validation labels are hand-assigned independently of the scoring formula; only measured numbers are published (`artifacts/validation`).
- D9 Pass thresholds Junior 0.60 / Middle 0.70 / Senior 0.75, upper sublevel at 0.85; matching weights 0.40 / 0.20 / 0.15 / 0.10 / 0.10. Both are initial tunable configuration, not scientific constants.
- D10 Grade tolerance in matching is ±1 internal sublevel (target M2 → M1, M2, S1), never ±1 public grade.
- D11 Contacts are released only by `invitations.service.contacts_for` for an ACCEPTED/CONTACTS_UNLOCKED invitation of the requesting employer; hiding is server-side.
- D12 MVP authentication: signed bearer tokens issued at registration (no passwords) plus demo

accounts when `DEMO_MODE=true`; Keycloak / FSP ID is the planned replacement behind the same dependency.

2026-10-10 — Core Assessment: 11 tasks per grade; CAT evaluated, not adopted

- Bank widened from 8 to 11 families per grade (33 total, 6–10 presets each; rubric `backend-core-2026.10.2`). Time limits 15/19/23 min.
- Computerised adaptive testing was tried on the existing bank (simulation, 20k synthetic candidates per grade, uniform ability, 11 items). The bank has one difficulty band per grade and no IRT calibration, so true CAT (item selection by information) is not defensible. The only sound variant is curtailment (stop when pass/fail is decided): pass/fail agreement with the full test was 100% and it saves 3.3 / 3.8 / 4.7 items (J/M/S) on average, but the sublevel (J1–S3), which needs the full score, becomes unreliable on a truncated attempt (and frozen variants + autosave assume a fixed set). **Not adopted**; the fixed 11-item test stays.

Demo guide

Start: `docker compose up --build` → http://localhost:3000 (API docs: http://localhost:8000/docs). Demo data is loaded automatically (`DEMO_MODE=true`).

Reset to the initial state: `docker compose exec backend python -m fsp_market.demo reset` (outside Docker: `python -m fsp_market.demo reset`). List accounts: `python -m fsp_market.demo accounts`.

Accounts (fixtures, no passwords): Alexey Demo (candidate with FSP history, `FSP-1001`), Mila Newcomer (candidate without FSP history, `FSP-2002`), Anna Verified (already VERIFIED), Hana Almost (NOT_VERIFIED Middle), Fedor Expensive (salary mismatch), Demo Corp (employer).

Canonical scenario and the decline path are in the root `README.md` (section "Демо-сценарий"). Automated proofs:

- `python scripts/smoke.py http://localhost:3000` — walks the whole canonical demo plus the decline path through the running stack;
- `tests/test_phase7_demo.py` — the same flow in-process, privacy sweep, reset;
- a headless-Chrome run of the UI flow was used during development (candidate → employer → accept → contacts → pool → validation page).

Validation page: header link "Валидация" (data from `artifacts/validation/latest.json`).

Тестовые роли и новые сценарии

Кандидат: регистрация → письмо/демо-ссылка → выход → повторный вход паролем или кодом. Работодатель: подбор по потребности или самостоятельное приглашение из полного пула (`need_id=null`); тип зарплаты GROSS/NET переносится из потребности и может быть изменён вручную. Контакты видны только отправителю после ACCEPTED. Проверено автоматически: `tests/test_phase11_auth_invites.py`, `scripts/smoke.py`.